
UNIVALI — Escola Politécnica — NEI

Curso de Ciência da Computação

Disciplina: Banco de Dados

Prof. Maurício Pasetto de Freitas, MSc.

Projeto e Implementação de Banco de Dados

Plataforma de Streaming de Filmes e Séries

Trabalho Avaliativo — M3

Autores:

Angelo Miguel Requenha

Bruno de Queiróz

Itajaí — SC, 2026

Sumário

Sumário	1
1. Definição do Problema	2
2. Projeto Conceitual (DER)	3
3. Projeto Lógico (normalizado)	4
4. Projeto Físico (MySQL)	5
5. Desenvolvimento (CRUD)	7
6. Repositório no GitHub	8

1. Definição do Problema

Este projeto tem como domínio uma **plataforma de streaming de filmes e séries**, no estilo de serviços como **Netflix** e **Prime Vídeo**. O problema central desse tipo de serviço é organizar, de forma consistente e confiável, um grande catálogo de conteúdo audiovisual e associá-lo aos usuários que o consomem. Sem um banco de dados bem modelado, torna-se inviável responder a perguntas essenciais para o negócio, como "quais títulos existem no catálogo?", "quais vídeos determinado usuário já assistiu?" ou "quem consumiu determinado conteúdo?". O público-alvo é o **usuário final**, que deseja acessar um acervo de vídeos e acompanhar seu histórico de visualização, além do **administrador da plataforma**, responsável por manter o catálogo e auditar as operações realizadas no sistema.

Para resolver esse problema, o sistema precisa manter o cadastro de **usuários** e de **vídeos** e registrar o **histórico** de quais usuários assistiram a quais vídeos — uma relação de muitos-para-muitos que exige uma entidade associativa. Adicionalmente, por se tratar de um serviço com múltiplos usuários e conteúdo sensível, é desejável manter uma trilha de **auditoria** das operações de criação e remoção de usuários e vídeos, garantindo rastreabilidade. Assim, a solução proposta contempla o cadastro e a manutenção (CRUD) das entidades principais, o relacionamento entre elas por meio de chaves estrangeiras e um mecanismo automático de registro de eventos, tudo persistido em um banco de dados relacional MySQL e operado por uma aplicação de linha de comando em Python.

Entidades principais

- **USUÁRIO (users)**: quem acessa a plataforma (cadastro, login e configurações).
- **VÍDEO (videos)**: os títulos do catálogo (filmes e séries), com categoria e URL.
- **HISTÓRICO (history)**: entidade associativa que liga usuários aos vídeos assistidos.
- **AUDITORIA (log_usuarios / log_videos)**: trilha automática de eventos de criação e exclusão, unificada pela view `vw_auditoria`.

2. Projeto Conceitual (DER)

O Diagrama Entidade-Relacionamento (DER) conceitual, em notação de Chen, representa o núcleo do domínio. Um **usuário** pode assistir a muitos **vídeos** e um vídeo pode ser assistido por muitos usuários: trata-se de um relacionamento **muitos-para-muitos** (N:N), materializado pelo relacionamento **ASSISTE**, que dá origem à entidade associativa *Histórico* (com o atributo *data_visto*).

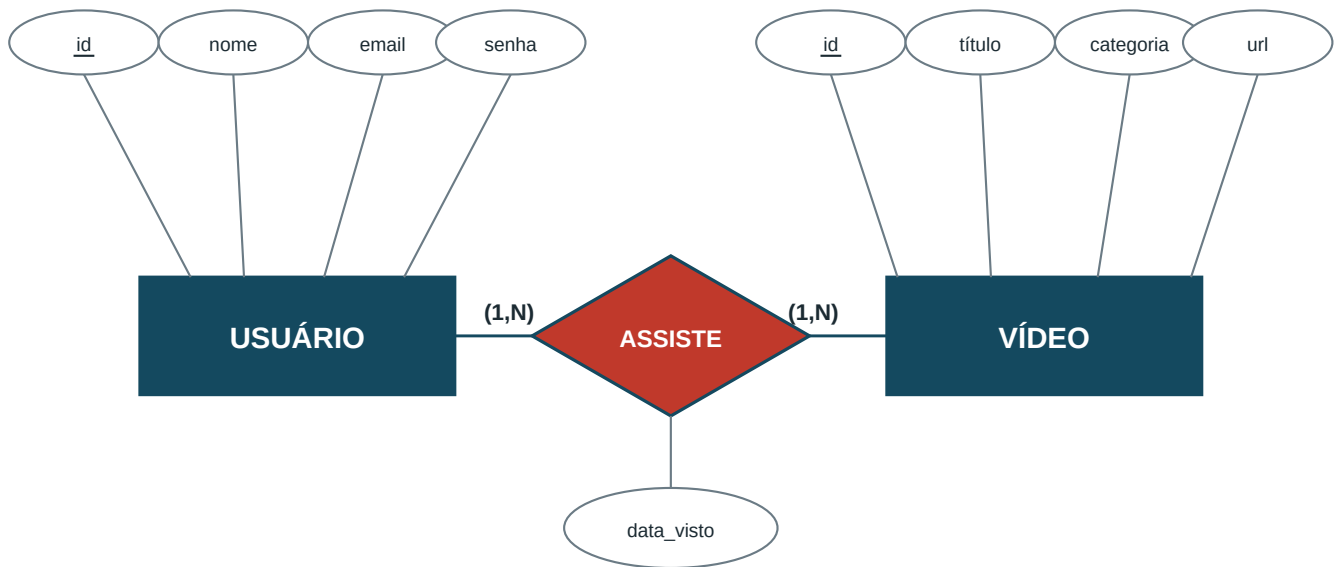


Figura 1 — DER conceitual (notação de Chen): Usuário × Vídeo.

De forma complementar, o sistema mantém uma **auditoria**: cada evento de inserção ou remoção de um usuário ou vídeo gera automaticamente um registro na entidade de log, conforme ilustrado abaixo. Esse comportamento é implementado no nível físico por meio de *gatilhos* (triggers).

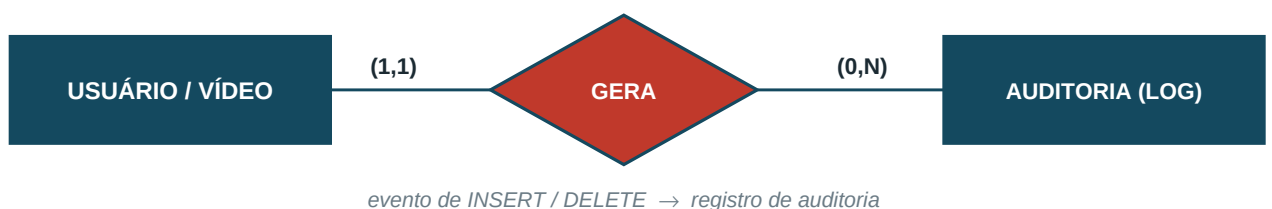


Figura 2 — Fragmento conceitual do subsistema de auditoria.

3. Projeto Lógico (normalizado)

O modelo lógico resolve o relacionamento N:N por meio da tabela associativa *history*, que armazena as chaves estrangeiras para *users* e *videos*. O esquema está normalizado até a **3ª Forma Normal (3FN)**: todos os atributos são atômicos (1FN); todos dependem integralmente da chave primária, que é simples em todas as tabelas (2FN); e não há dependências transitivas entre atributos não-chave (3FN).

Estrutura relacional (formato textual)

- **users** (id, username, email, password, created_at) — email UNIQUE.
- **videos** (id, title, description, category, url, created_at).
- **history** (id, *user_id*, *video_id*, date_watched) — *user_id* → users(id), *video_id* → videos(id).
- **log_usuarios** (id, usuario_id, acao, data_evento).
- **log_videos** (id, video_id, acao, data_evento).

Legenda: sublinhado = chave primária (PK); *itálico* = chave estrangeira (FK).

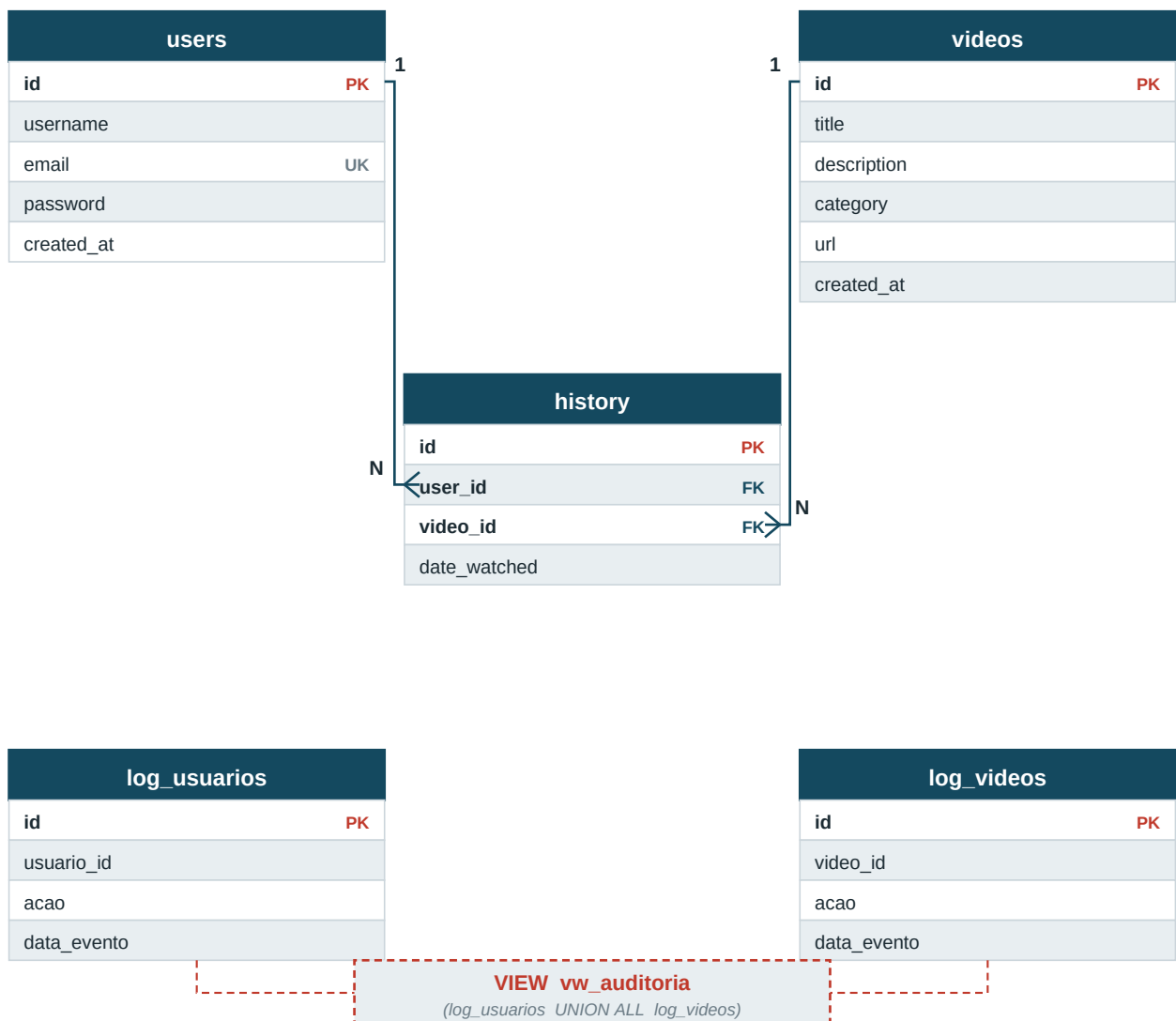


Figura 3 — Diagrama Relacional (DR) do projeto lógico normalizado.

4. Projeto Físico (MySQL)

A seguir, o código SQL de criação do esquema e das tabelas no MySQL, incluindo as chaves primárias, a restrição de unicidade do e-mail e as chaves estrangeiras da tabela *history*. O mesmo diagrama relacional da Figura 3 corresponde a esta implementação física.

4.1 Criação do esquema e das tabelas

```
CREATE DATABASE IF NOT EXISTS streaming;
USE streaming;

CREATE TABLE IF NOT EXISTS users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(255) NOT NULL,
    email VARCHAR(255) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS videos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(255) NOT NULL,
    description TEXT,
    category VARCHAR(255),
    url VARCHAR(255) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS history (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    video_id INT NOT NULL,
    date_watched TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id),
    FOREIGN KEY (video_id) REFERENCES videos(id)
);

CREATE TABLE IF NOT EXISTS log_usuarios (
    id INT AUTO_INCREMENT PRIMARY KEY,
    usuario_id INT,
    acao VARCHAR(50),
    data_evento DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS log_videos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    video_id INT,
    acao VARCHAR(50),
    data_evento DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

4.2 Gatilhos de auditoria e view de leitura

Os *triggers* populam automaticamente as tabelas de log a cada INSERT/DELETE; a view *vw_auditoria* unifica os dois logs para consulta e relatório.

```
-- Gatilhos (triggers): auditam INSERT/DELETE em users e videos
DELIMITER //
CREATE TRIGGER trg_usuario_criado AFTER INSERT ON users
FOR EACH ROW BEGIN
    INSERT INTO log_usuarios (usuario_id, acao)
    VALUES (NEW.id, 'USUARIO_CRIADO');
END//
CREATE TRIGGER trg_usuario_deletado AFTER DELETE ON users
FOR EACH ROW BEGIN
    INSERT INTO log_usuarios (usuario_id, acao)
    VALUES (OLD.id, 'USUARIO_DELETADO');
END//
CREATE TRIGGER trg_video_criado AFTER INSERT ON videos
FOR EACH ROW BEGIN
    INSERT INTO log_videos (video_id, acao)
    VALUES (NEW.id, 'VIDEO_CRIADO');
END//
CREATE TRIGGER trg_video_deletado AFTER DELETE ON videos
FOR EACH ROW BEGIN
    INSERT INTO log_videos (video_id, acao)
    VALUES (OLD.id, 'VIDEO_DELETADO');
END//
DELIMITER ;

-- View que unifica os dois logs para leitura/relatorio
CREATE OR REPLACE VIEW vw_auditoria AS
SELECT 'USUARIO' AS tipo, usuario_id AS referencia_id, acao, data_evento
FROM log_usuarios
UNION ALL
SELECT 'VIDEO' AS tipo, video_id AS referencia_id, acao, data_evento
FROM log_videos;
```

4.3 População das tabelas (dados de exemplo)

```
-- Populacao das tabelas com dados de exemplo
INSERT INTO users (username, email, password) VALUES
('admin', 'admin@stream.com', 'admin'),
('joao', 'joao@email.com', '1234'),
('maria', 'maria@email.com', 'abcd');

INSERT INTO videos (title, description, category, url) VALUES
('Matrix', 'Ficcao cientifica classica', 'Filme', 'https://cdn.stream.com/matrix'),
('Stranger Things', 'Serie de misterio e terror', 'Serie', 'https://cdn.stream.com/stranger'),
('Interestelar', 'Viagem espacial e buracos de minhoca', 'Filme', 'https://cdn.stream.com/interstellar'),
('Breaking Bad', 'Drama sobre um professor de quimica', 'Serie', 'https://cdn.stream.com/breakingbad');

-- date_watched e preenchido automaticamente (DEFAULT CURRENT_TIMESTAMP)
INSERT INTO history (user_id, video_id) VALUES
(2, 1),
(2, 3),
(3, 2),
(3, 4);
```

5. Desenvolvimento (CRUD)

A aplicação foi desenvolvida em **Python 3**, com persistência em **MySQL** via o driver `mysql-connector-python`. A interface é de **linha de comando** (console), com navegação por setas do teclado (biblioteca `readchar`). O acesso é feito por login: o usuário `admin` gerencia o catálogo e a auditoria; os demais usuários listam vídeos e gerenciam a própria conta.

5.1 Arquitetura em camadas

- **config/** — conexão com o banco (`connect_db`).
- **create_db/** — criação automática do banco a partir do script `streaming.sql`.
- **src/models/** — classes de domínio (`User`, `Video`, `History`, `Audit`).
- **src/repositories/** — acesso a dados; concentra **todo o SQL**.
- **src/services/** — regras de negócio (ex.: login, troca de senha).
- **src/views/** — menus e interação com o usuário no terminal.

5.2 Operações CRUD por entidade

Entidade	Create	Read	Update	Delete
users	sim	sim	sim	sim
videos	sim	sim	sim	sim
history	sim	sim	—	sim
auditoria	auto (trigger)	sim (view)	—	—

5.3 Exemplo de código — CRUD de usuário

O SQL é escrito manualmente e parametrizado (uso de `%s`, que previne SQL Injection). **Não se utiliza ORM**, portanto os comandos SQL são explícitos no código, dispensando comentários de abstração exigidos pelo enunciado nesse caso.

```
# src/repositories/user_repository.py (SQL escrito a mao, sem ORM)

def create_user(self, name, email, password):
    # CREATE
    query = "INSERT INTO users (username, email, password) VALUES (%s, %s, %s)"
    cursor.execute(query, (name, email, password))
    self.connection.commit()

def get_all_users(self):
    # READ
    query = "SELECT id, username, email, password, created_at FROM users"
    cursor.execute(query)
    return [User(*row) for row in cursor.fetchall()]

def update_user(self, user_id, name=None, email=None, password=None):
    # UPDATE
    query = "UPDATE users SET username = %s, email = %s, password = %s WHERE id = %s"
    cursor.execute(query, (name, email, password, user_id))
    self.connection.commit()

def delete_user_by_id(self, user_id):
    # DELETE
    query = "DELETE FROM users WHERE id = %s"
    cursor.execute(query, (user_id,))
    self.connection.commit()
```


6. Repositório no GitHub

O código-fonte completo, incluindo o script SQL de criação do banco e das tabelas, está publicado em repositório **público** no GitHub, atendendo ao Item 2 da Parte 02:

`https://github.com/Angelo-Requenha/M3-Banco\_de\_dados`